

This is a great project. You have a very clear scope, defined constraints, and a good understanding of the tools you already have.

Creating editors for data-driven games like Battletech is a very common task. The core challenge you face isn't actually the graphics; it's managing the sheer volume and nested complexity of the JSON data, and then creating a UI that makes editing that data less painful than using Notepad++.

Given your requirements—specifically the need to parse massive amounts of JSON, create complex UIs with dropdowns and text fields, *and* render 2D images (DDS) simultaneously—I have a strong recommendation, and then a solid backup alternative based on your existing C++ setup.

Here is an analysis of the best paths forward to achieve your "dumb little editor" quickly.

The Critical Challenge: DDS Files and JSON

Before choosing a language, we have to address the two biggest hurdles:

1. **JSON Handling:** Battletech's JSON definitions (MechDefs, ChassisDefs) are deep and complex. In C++, managing deeply nested JSON objects, iterating over dynamic arrays, and handling string conversions can become tedious very quickly. In JavaScript, JSON is native; it just works.
2. **DDS Images:** DDS (DirectDraw Surface) is a texture format meant for GPUs. Standard 2D UI libraries (like standard Windows Forms, or web browsers) generally *do not* know how to display them natively. You almost always need a dedicated library to decode the DDS into a standard raw bitmap (RGBA) before you can display it on screen.

Recommendation 1: The Modern "Easy" Path (JavaScript/Node.js + Electron)

Even though you have a C++ background, I strongly recommend this path for this specific project. It is the fastest way to build complex data-entry forms that also need to display images.

Concept:

Use Electron. Electron takes Node.js (for backend file access) and Chromium (for rendering the UI with HTML/CSS) and bundles them into a standalone executable folder.

Why it fits your needs:

- **JSON is Native:** Reading a MechDef file is literally `JSON.parse(fs.readFileSync('mechdef_xyz.json'))`. You immediately have a usable Javascript object. No libraries, no complex typing.
- **UI is Easy:** Creating dropdowns for weapons lists, checkboxes for equipment, and layout grids for mech slots is incredibly fast using standard HTML/CSS.
- **Offline/Standalone:** It packages into a folder with an .exe that requires no internet and runs anywhere.
- **The DDS Solution:** Because Electron runs Node.js in the background, you can use a

Node.js package to read a DDS file from the game folder, convert it in memory to a standard PNG format, and pass that to the HTML front-end to display in a standard `<img>` tag.

The Workflow:

1. You set up an Electron project (using something like Electron Forge for easy scaffolding).
2. Your "backend" JS script asks the user for the Battletech install directory.
3. It recursively scans directories, ingesting vanilla and mod JSON files into memory objects.
4. Your "frontend" HTML/JS builds forms based on that data.
5. When you select a chassis, the backend reads the corresponding DDS portrait, converts it, and sends it to the frontend to display.
6. When you click save, you `JSON.stringify()` your data object and write it to your mod folder.

Pros: Fastest development time for UI and JSON heavy tasks.

Cons: The resulting executable folder is large (usually 100MB+ base size) because it contains a stripped-down web browser.

Recommendation 2: The C++ Path (Dear ImGui + SFML/SDL)

If you prefer to stick to C++ and your existing MSYS2/Clang64 workflow, this is the best route.

Crucial Advice: Do not use Irrlicht for this. Irrlicht is a 3D engine with a basic GUI tacked on. Building complex data-entry forms in Irrlicht's native GUI system is incredibly verbose and difficult to maintain.

Instead, use an "Immediate Mode" GUI library called **Dear ImGui**.

Concept:

Dear ImGui is the industry standard for making internal tools and editors in C++. It draws UI widgets every single frame. It is incredibly fast to set up complex windows with tabs, inputs, and lists. You need a "backend" to open a window and handle inputs, and a "renderer" to draw the pixels.

Given your background, you could use **SFML** (which you know) to open the window and handle mouse input, and let ImGui handle the actual UI drawing on top of it.

The Toolchain:

- **Compiler:** Your existing MSYS2 Clang64.
- **IDE:** Codeblocks.
- **Window/Input Backend:** SFML (since you have experience with it) or SDL2.
- **UI Library:** Dear ImGui (you just include its .cpp and .h files directly in your project).
- **JSON Library:** nlohmann/json (The modern standard C++ JSON library. It's header-only, just drop the file in).

The DDS Problem in C++:

This is the hardest part of the C++ route. SFML does not load DDS files natively. You will need a third-party C++ library specifically designed to decode DDS files (like `tinyddsloader` or a

specific configuration of stb_image) into raw pixel data, and then load that raw data into an SFML sf::Texture.

Pros: Uses your preferred language and existing toolchain. Resulting executable is tiny and fast. Dear ImGui is excellent for tweaking data values.

Cons: JSON handling in C++ using nlohmann/json, while good, is vastly more verbose than JavaScript. Solving DDS loading in C++ will require integrating another obscure library.

Summary & Final Recommendation

You want a "dumb little editor" that you don't want to take forever to make.

The friction in this project is not rendering pixels; it's handling the massive spiderweb of Battletech's JSON data structures and building the UI forms to act on them.

1. **Go with the Electron (JS) route** if your priority is getting the editor finished fast and having an easy time building the UI forms and handling JSON data. The trade-off is a larger executable size.
2. **Go with the C++ (Dear ImGui + SFML + nlohmann/json) route** if your priority is keeping a small executable footprint, sticking strictly to C++, and you are willing to spend extra time wrestling with C++ JSON syntax and finding a DDS loading solution.

Given your description, I honestly believe the **Electron/JS** route will save you dozens of hours of frustration compared to the C++ route for this specific type of application.